Porównanie modeli równoległych obliczeń inkrementalnych PSAC i Query w zastosowaniu do kompilatorów

Jakub Bartmiński Piotr Oleszczuk Piotr Trzaskowski

Projekt Badawczy, rok akademicki 2025/2026
Wydział Matematyki, Informatyki i Mechaniki
Uniwersytet Warszawski
Opiekun: Aleksy Schubert

Streszczenie

Współbieżny kompilator inkrementalny musi jednocześnie korzystać z wielu rdzeni i szybko przeliczać wynik po drobnej edycji kodu. Porównaliśmy dwa modele obliczeń, które dają obie te rzeczy naraz: PSAC (*Parallel Self-Adjusting Computation*), oparty na propagacji zmian po zapisanym śladzie wykonania, oraz Query, autorski framework zapytań używany przez kompilator języka Duckling, rozwijanego przez firmę DuckType, oparty na grafie zapytań z cache'em wyników. Napisaliśmy dwa benchmarki syntetyczne, każdy celowo dopasowany do innego modelu, oraz dwa pełne kompilatory inkrementalne, które dzielą ten sam front end i ten sam generator kodu, a różnią się wyłącznie silnikiem inkrementalnym. W benchmarkach syntetycznych różnice były duże: PSAC okazał się do 17 razy szybszy przy płytkich zależnościach drzewiastych, a Query do 2,5 raza szybsze przy gęstych zależnościach współdzielonych. W prawdziwym kompilatorze ta przewaga w dużej mierze zniknęła, bo przy pełnej kompilacji różnice między silnikami zmieściły się w 5–20%, a przy aktualizacji inkrementalnej były nieistotne. Znacznie większy wpływ na czas przebudowy miała granularność śledzonych zależności, której poprawa dała efekt o rząd wielkości większy niż jakakolwiek różnica między silnikami. Wnioski wykorzystaliśmy do optymalizacji kompilatora Duckling, gdzie zmiana alokatora pamięci i usunięcie globalnej blokady chroniącej tablicę internowanych napisów dały odpowiednio do 50% i 10–20% przyspieszenia.

1 Wprowadzenie i pytania badawcze

Od kompilacji dużego projektu oczekuje się dziś dwóch rzeczy naraz. Po pierwsze, ma korzystać ze wszystkich dostępnych rdzeni, bo czas pełnej kompilacji przekłada się wprost na koszt integracji ciągłej. Po drugie, ma reagować na pojedynczą edycję tak szybko, żeby wydawało się to natychmiastowe, bo z tego samego potoku korzysta serwer języka obsługujący edytor. Te wymagania nie są niezależne: zrównoleglenie opłaca się tylko wtedy, gdy jest co liczyć, a inkrementalność ma sens tylko wtedy, gdy ponowne użycie wyników nie kosztuje więcej niż samo policzenie ich od nowa.

Są dwa znane podejścia do tego problemu. Pierwsze to *self-adjusting computation*: program wykonuje się raz i zapisuje ślad swojego wykonania wraz z informacją o tym, które dane wejściowe czytał. Po zmianie wejścia system przepycha tę zmianę przez ślad i wykonuje ponownie tylko te fragmenty, które od niej zależą. Model PSAC [1] rozszerza to na programy równoległe i układa ślad w drzewo szeregowo-równoległe. Drugie podejście to *grafy zapytań*: obliczenie dzieli się na nazwane, czyste zapytania, których wyniki trafiają do cache'u, a zależności odkrywane są w trakcie wykonania. Do tej grupy należy Query, framework używany w kompilatorze języka Duckling. Autorska jest przy tym konkretna implementacja tego frameworka, a nie sam pomysł grafu zapytań z cache'em.

Pytanie, które postawiliśmy sobie na początku, brzmiało prosto: który z tych dwóch modeli lepiej nadaje się na silnik kompilatora inkrementalnego? Rozbiliśmy je na cztery pytania badawcze:

RQ1 Który silnik ma mniejszy narzut własny, w oderwaniu od realnej pracy?

RQ2 Jak oba skalują się wraz z liczbą wątków?

RQ3 Czy przewaga zmierzona na benchmarkach syntetycznych przenosi się na prawdziwy kompilator?

RQ4 Co bardziej decyduje o czasie przebudowy inkrementalnej: wybór silnika czy granularność śledzonych zależności?

2 Dwa modele obliczeń inkrementalnych

PSAC. Program w modelu PSAC czyta dane wejściowe wyłącznie przez specjalne, modyfikowalne komórki. Każdy odczyt komórki tworzy w śladzie wykonania węzeł odczytu, a każde rozwidlenie równoległe tworzy węzeł równoległy, dzięki czemu ślad ma postać drzewa szeregowo-równoległego. To drzewo nie jest tylko planem wykonania, ale też zapisaną strukturą zależności. Zapis nowej wartości do komórki oznacza jako nieaktualne te węzły odczytu, które ją czytały. Propagacja przechodzi drzewo i wykonuje ponownie tylko nieaktualne poddrzewa, rozwidlając się przy tym tak samo jak wykonanie pierwotne. Wykonaniem steruje scheduler fork-join z kradzieżą zadań (work stealing): każde rozwidlenie odkłada jedną gałąź na lokalną kolejkę wątku, a wolne wątki podbierają pracę z kolejek losowo wybranych sąsiadów, więc jedna kosztowna gałąź nie blokuje reszty. PSAC nie zapamiętuje jednak wyników po tym, co dane podobliczenie liczy. Jeżeli dwa różne miejsca programu liczą to samo, praca wykona się dwa razy.

Query. W modelu Query obliczenie dzieli się na zapytania, z których każde ma swój klucz. Wynik zapytania trafia do cache'u, więc dwa niezależne zapytania o ten sam klucz wykonają pracę tylko raz. Zależności deklaruje się w trakcie wykonania: funkcja obsługująca zapytanie odpytuje *side inputy*, czyli liście grafu, których kluczem jest sama wartość wejściowa. Po zmianie wejścia stary liść znika z grafu, a framework przechodzi graf zależności w odwrotnym kierunku i kasuje dokładnie te węzły, które od niego zależały. Ponowne wykonanie następuje dopiero przy kolejnym odpytaniu. Za równoległość odpowiada pula zadań, która wykonuje niezależne zapytania jednocześnie.

Różnice zbiera tabela 1. Z punktu widzenia kompilatora najważniejsze są dwie: obecność cache'u wyników oraz to, że w PSAC struktura równoległości i struktura zależności to jedno i to samo drzewo.

Tabela 1: Porównanie modeli PSAC i Query.

Cecha	PSAC	Query
Kierunek wykonania	przepychanie zmiany (push)	wykonanie na żądanie (pull)
Struktura ponownego użycia	zapisany ślad wykonania w postaci drzewa szeregowo-równoległego	graf zapytań z cache'em wyników
Jednostka ponownego użycia	węzeł odczytu w śladzie	zapytanie o danym kluczu
Deklaracja zależności	odczyt modyfikowalnej komórki wewnątrz zakresu dynamicznego	odpytanie side inputu
Reakcja na zmianę wejścia	zapis unieważnia czytelników, propagacja przechodzi nieaktualne poddrzewa	unieważnienie po odwrotnym grafie zależności, przeliczenie na żądanie
Współdzielone podobliczenia	brak cache'u, praca może się dublować	cache gwarantuje jednokrotne wykonanie
Model równoległości	fork-join z kradzieżą zadań, ślad jest zarazem planem wykonania	pula zadań, zapytania zlecane jako punkty wejścia
Nieudane podobliczenie	zależność od hasha całego programu	zależność od hasha całego programu

3 Metodologia

Eksperyment prowadziliśmy na dwóch poziomach: najpierw na syntetycznych obliczeniach, potem na dwóch napisanych przez nas kompilatorach.

Benchmarki syntetyczne. Napisaliśmy dwa syntetyczne obliczenia, każde celowo dopasowane do innego modelu, żeby zobaczyć, jak duża może być przewaga każdego z nich w najlepszym dla niego

przypadku.

ArraySum to drzewo przedziałowe nad tablicą liczb: wartość węzła jest sumą wartości jego dzieci, a liśćmi są pojedyncze elementy tablicy. Zależności są tu płytkie i drzewiaste, a każdy zakres `[l,r)` występuje dokładnie raz, więc nie ma współdzielonych poddrzew i cache Query nie ma czego ponownie użyć, wnosi tylko narzut. Aktualizacja zmienia zadany procent elementów tablicy, a zmiana propaguje się ścieżką od liścia do korzenia, dotykając logarytmicznie mało węzłów. Jest to układ możliwie korzystny dla PSAC.

DAG to warstwowy graf skierowany: N warstw po M wierzchołków, krawędzie tylko między sąsiednimi warstwami, a wartość wierzchołka to jego waga plus suma wartości poprzedników. Zależności są gęste i współdzielone, bo jeden wierzchołek jest poprzednikiem wielu, więc jego zmiana wpływa na dużą część grafu. Cache Query sprawia, że każdy wierzchołek liczony jest dokładnie raz, podczas gdy PSAC wykonuje tu część pracy podwójnie. Ten układ jest z kolei korzystny dla Query.

Oba mierzyliśmy w dwóch trybach: samo obliczenie oraz obliczenie wraz z aktualizacją inkrementalną. Parametrem *workload* sterowaliśmy ilością sztucznej pracy przypadającej na jeden węzeł. Ten parametr jest kluczowy: przy wartości zero mierzymy czysty narzut frameworka, a przy wyższych wartościach narzut rozmywa się w prawdziwej pracy, co odpowiada sytuacji w kompilatorze.

Kompilatory. Napisaliśmy dwa kompilatory inkrementalne dla tego samego podzbioru języka Duckling: funkcje z rekurencją, podstawowe typy, klasy z polami, tablice w stylu C, przestrzenie nazw, stałe, zmienne globalne i import wielu plików. Oba kompilują każdą funkcję do osobnego pliku obiektowego, równolegle, i linkują wynik jednym wywołaniem systemowego linkera. Nazwaliśmy je *psacklinger* (silnik PSAC) oraz *querylinger* (silnik Query) i tych nazw używamy w całym raporcie.

Najważniejsze w tym porównaniu jest to, że oba nasze kompilatory dzielą cały front end oraz cały generator kodu: ten sam parser, tę samą rejestrację deklaracji globalnych, te same tablice symboli i to samo generowanie kodu LLVM. Różnią się wyłącznie tym, jak decydują, co przekompilować. Dzięki temu każdą zmierzoną różnicę można przypisać silnikowi inkrementalnemu, a nie jakości implementacji front endu.

Jako punkt odniesienia mierzyliśmy też *duckc*, czyli prawdziwy kompilator języka Duckling, również oparty na Query. Nie jest to przeciwnik w tej samej wadze: duckc obsługuje cały język, ma znacznie bogatszy front end, dwa backendy generujące kod, diagnostykę, obsługę pakietów i pełną inkrementalność na poziomie pojedynczych elementów drzewa składniowego, podczas gdy nasze dwa kompilatory to zabawki obsługujące wąski podzbiór języka. Porównanie z nim nie mówi więc nic o jakości implementacji, a jedynie pokazuje, ile da się wycisnąć, gdy zadanie zawęzi się do jednej fazy.

Żeby porównanie granularności miało sens, oba nasze kompilatory muszą zachowywać się identycznie. Sprawdza to wspólny zestaw około 108 testów typu end-to-end: każdy kompiluje program pełnym potokiem, uruchamia powstały plik wykonywalny i sprawdza kod wyjścia. Testy inkrementalne dodatkowo sprawdzają, ile dokładnie funkcji zostało przekompilowanych po danej edycji.

Fragmenty kodu 1 i 2 pokazują, jak w obu implementacjach deklarowana jest zależność.

```
psac_function(compileUnitPsac, State* s, FunctionUnit* unit) {
    psac_dynamic_context({ // zakres dynamiczny, sledzi co obliczenie przeczytalo
        _psac_context_mods.push_back(&unit->body_mod); // hash ciala funkcji
        _psac_context_mods.push_back(&s->structural_mod); // hash importow i nazw
        // Recorder dopisze komorki symboli, po ktore siegnie generator kodu.
        DynamicDepRecorder recorder(_psac_context_mods, *s, s->ctx);
        compileOneFunction(s->ctx, unit, &recorder);
    });
}
```

Fragment kodu 1: Kompilacja jednej funkcji jako osobne obliczenie PSAC. Zakres dynamiczny zbiera komórki odczytane w trakcie wykonania, więc zbiór zależności wyliczany jest od nowa przy każdym przebiegu.

```
// Side input, czyli lisc grafu zapytan. Jego kluczem jest hash wartosci.
DECLARE_QUERY_SIDE_INPUT(SigInput, HashInputKey);

void SymbolUseRecorder::onFunctionUse(const sym::Symbol& symbol) override {
```

```
    // Odpytanie liscia zapisuje w grafie krawedz zaleznosci od tej sygnatury.
    query_ctx.query<SigInput>({ .hash = functionSigInputHash(program_ctx, symbol) });
}
```

Fragment kodu 2: Rejestracja zależności w wersji Query. Metoda `onFunctionUse` wywoływana jest w momencie, w którym generator kodu sięga po sygnaturę funkcji, więc zależność powstaje dokładnie tam, gdzie symbol jest naprawdę używany.

Środowisko pomiarowe. Wszystkie pomiary, i syntetyczne, i kompilatorów, wykonaliśmy na tej samej maszynie: procesor Intel Core Ultra 7 165H (22 procesory logiczne), 63719 MiB pamięci RAM, system Linux 6.19.6-arch1-1, kompilator g++-15, kompilacja z flagami `-O3 -DNDEBUG`. Czasy mierzyliśmy narzędziem `hyperfine`, które mierzy czas całego procesu, wykonuje przebiegi rozgrzewające, powtarza pomiar wielokrotnie dla każdego zestawu parametrów i podaje średnią wraz z odchyleniem standardowym. Benchmarki syntetyczne przeszły pełne przeszukiwanie siatki parametrów: 3 przebiegi rozgrzewające i około 48 powtórzeń na punkt pomiarowy, 384 punkty pomiarowe dla ArraySum i 192 dla DAG. Benchmark kompilatorów używał 5 powtórzeń na wariant.

4 Wyniki: benchmarki syntetyczne

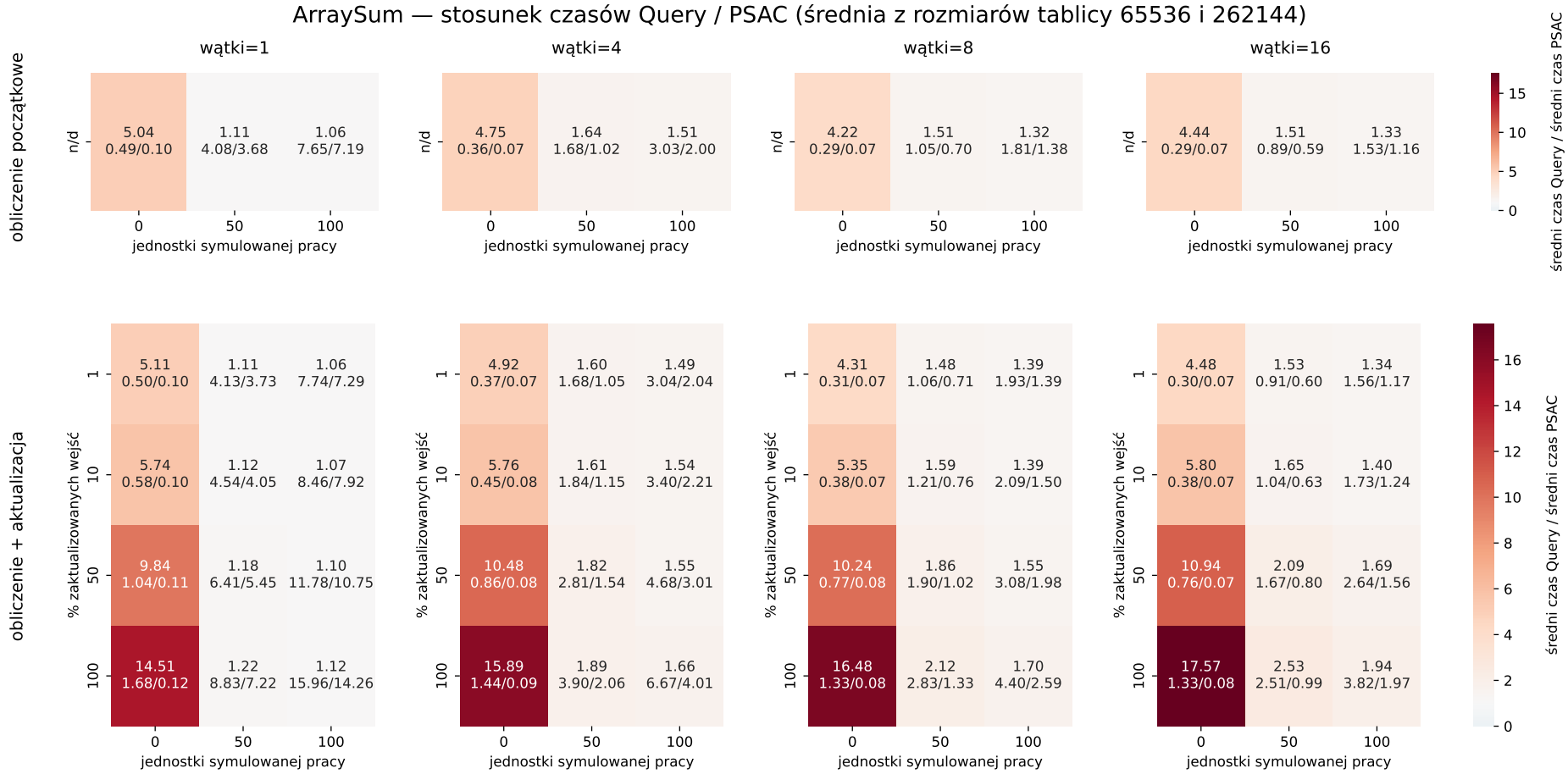
Siatka parametrów obejmowała cztery wymiary. Pierwszy to tryb: samo obliczenie albo obliczenie wraz z aktualizacją. Drugi to procent zaktualizowanych wejść, czyli jaka część danych zmienia się przed przeliczeniem inkrementalnym: 1%, 10%, 50% albo 100% elementów tablicy w ArraySum i wierzchołków w DAG-u. Trzeci to workload, czyli liczba jednostek sztucznej pracy wykonywanej w każdym węźle obliczenia, gdzie jedna jednostka to tysiąc obrotów pustej pętli. Wartość 0 oznacza brak pracy, więc mierzony jest sam narzut frameworka, a wartości 50 i 100 to pięćdziesiąt i sto tysięcy obrotów pętli na węzeł, czyli tyle pracy, że narzut przestaje dominować. Czwarty wymiar to liczba wątków: 1, 4, 8 albo 16. Tablica w ArraySum miała rozmiar 65536 albo 262144, a graf w DAG-u 50 warstw po 100 wierzchołków i 10000 krawędzi. Wyniki podajemy jako stosunek średniego czasu Query do średniego czasu PSAC, więc wartość powyżej jedności oznacza, że PSAC jest szybszy. Pełne wyniki przedstawiają rysunki 1 i 2.

ArraySum. Przy zerowym workloadzie, czyli na czystym narzucie frameworka, Query jest 4–5 razy wolniejsze od PSAC-a: stosunek wynosi 5,04 przy jednym wątku, 4,75 przy czterech, 4,22 przy ośmiu i 4,44 przy szesnastu. Sam graf zapytań kosztuje po prostu więcej niż lekki ślad PSAC-a. Wraz ze wzrostem sztucznej pracy przewaga maleje: przy workloadzie 50 stosunek mieści się w przedziale 1,11–1,64, a przy workloadzie 100 w przedziale 1,06–1,51. Oba silniki przyspieszają wraz z liczbą wątków.

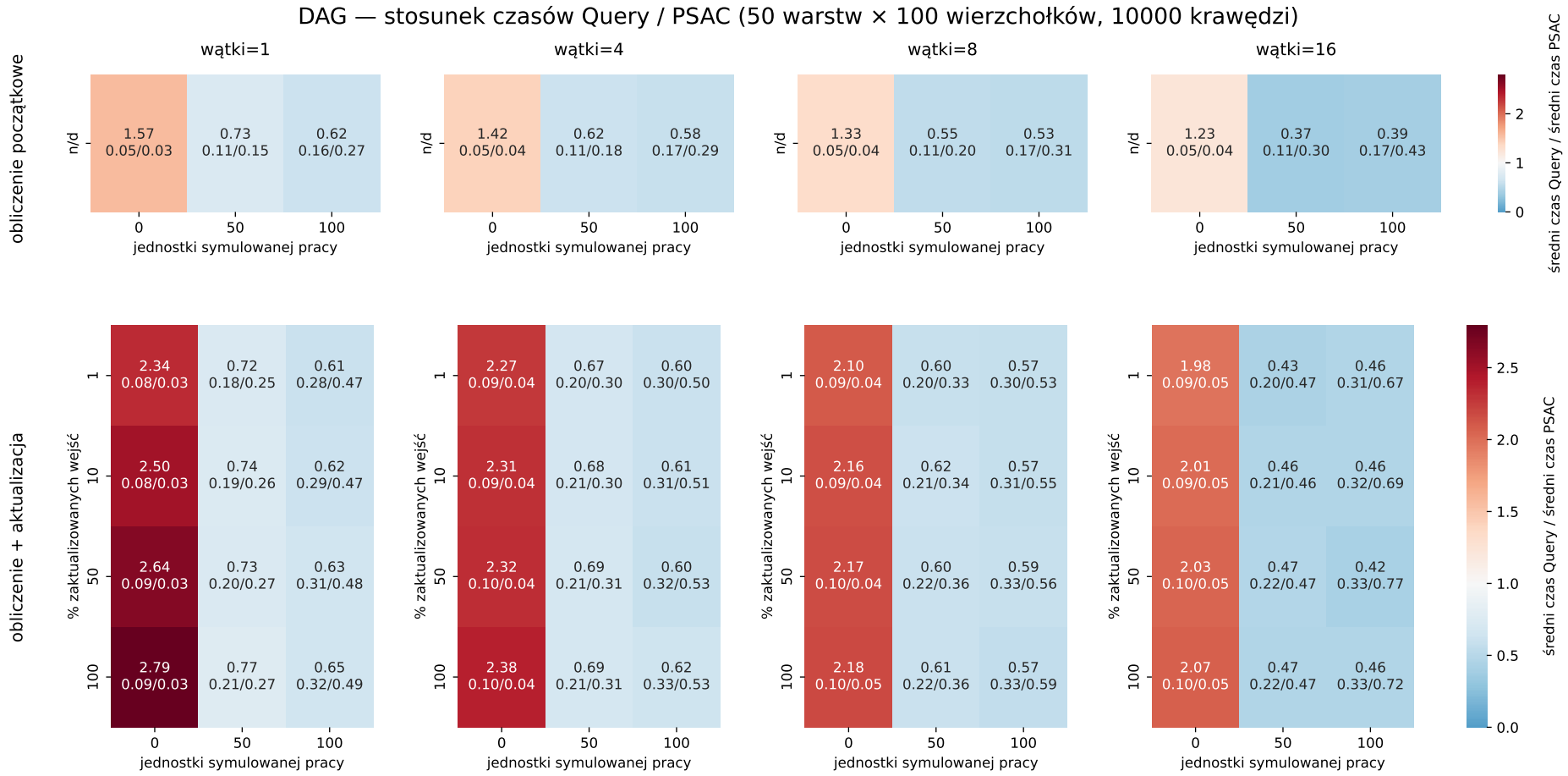
W trybie z aktualizacją różnica jest znacznie większa. Przy zerowym workloadzie i jednym procencie zmienionych elementów Query jest około pięć razy wolniejsze (5,11 przy jednym wątku, 4,48 przy szesnastu), a przy stu procentach zmienionych elementów już 14–18 razy wolniejsze (14,51 przy jednym wątku, 17,57 przy szesnastu). Powód widać w liczbach bezwzględnych: propagacja PSAC-a po prostym drzewie jest prawie darmowa, bo wzrost procentu aktualizacji z 1 do 100 zmienia jego czas tylko z 0,10 s na 0,12 s, podczas gdy Query przy tej samej zmianie przechodzi z 0,50 s na 1,68 s. Przy workloadzie 100 wyniki znów się wyrównują i Query jest wolniejsze 1,12–1,94 raza.

DAG. W tym benchmarku kolejność się zmienia. Na czystym narzucie PSAC dalej jest lżejszy, ale przewaga jest mała i maleje z liczbą wątków: 1,57 przy jednym wątku, 1,42 przy czterech, 1,33 przy ośmiu i 1,23 przy szesnastu. Gdy tylko pojawia się prawdziwa praca, wygrywa Query: przy workloadzie 50 stosunek spada do 0,73–0,37, a przy workloadzie 100 do 0,62–0,39. Odpowiada za to cache: każdy wierzchołek liczony jest dokładnie raz, podczas gdy PSAC przy bardziej złożonej strukturze obliczeń robi tę samą pracę wielokrotnie.

Skalowalność wygląda tu inaczej niż w ArraySum. Przy workloadzie 100 czas PSAC-a *rośnie* z 0,27 s na jednym wątku do 0,43 s na szesnastu, podczas gdy czas Query zostaje praktycznie płaski, zmieniając się z 0,16 s na 0,17 s. Nasza hipoteza, której nie sprawdziliśmy pomiarowo, jest taka,



Rysunek 1: ArraySum: stosunek średniego czasu Query do średniego czasu PSAC. Wiersze odpowiadają trybowi pomiaru, kolumny liczbie wątków. W każdej komórce u góry podany jest stosunek czasów, a pod nim czas Query i czas PSAC w sekundach. Kolor koduje ten sam stosunek: odcienie czerwieni oznaczają przewagę PSAC-a, odcienie błękitu przewagę Query, a biel remis. W trybie samego obliczenia oś procentu aktualizacji zwinęliśmy do jednego wiersza, bo nic się tam nie przelicza i wszystkie jej wartości są takie same. Wyniki są uśrednione po obu rozmiarach tablicy.



Rysunek 2: DAG: stosunek średniego czasu Query do średniego czasu PSAC. Układ i sposób odczytu jak na rysunku 1. Widać zmianę znaku przewagi wraz ze wzrostem sztucznej pracy oraz to, że procent aktualizowanych wierzchołków prawie nie wpływa na wynik.

że PSAC słabo radzi sobie z głębokimi zależnościami między danymi: długie łańcuchy zależności ograniczają dostępną równoległość, a koszt utrzymania drzewa śladu rośnie szybciej niż zysk z dodatkowych wątków.

W trybie z aktualizacją przy zerowym workloadzie PSAC jest szybszy w stosunku 1,98–2,79, przy czym stosunek ten prawie nie zależy od procentu aktualizacji (2,34 dla jednego procentu i 2,79 dla stu, przy jednym wątku). Przy workloadzie 50 i 100 przewaga wraca do Query, ze stosunkiem 0,42–0,77. Kontrast z ArraySum jest wyraźny: w gęstym grafie nawet mała zmiana wejścia unieważnia dużą jego część, więc oba frameworki i tak muszą przeliczyć sporo, przez co procent aktualizowanych wejść przestaje mieć znaczenie.

Zestawione razem, oba benchmarki pokazują, że PSAC wypada lepiej przy płytkich i prostych zależnościach, a Query przy gęstych i złożonych, czyli takich, jakie występują w kompilatorze. Przewaga Query rośnie przy tym z liczbą wątków i z ilością prawdziwej pracy przypadającej na węzeł.

5 Wyniki: kompilatory

Korpusem były dwa pliki wygenerowane automatycznie tym samym generatorem kodu, po około 8000 linii każdy. Różnią się wyłącznie liczbą funkcji: `duck1` zawiera ich 352, a `duck2` 153, więc przy zbliżonej łącznej długości funkcje w `duck2` są średnio ponad dwukrotnie dłuższe i jednostek pracy do zrównoleglenia jest w nim mniej.

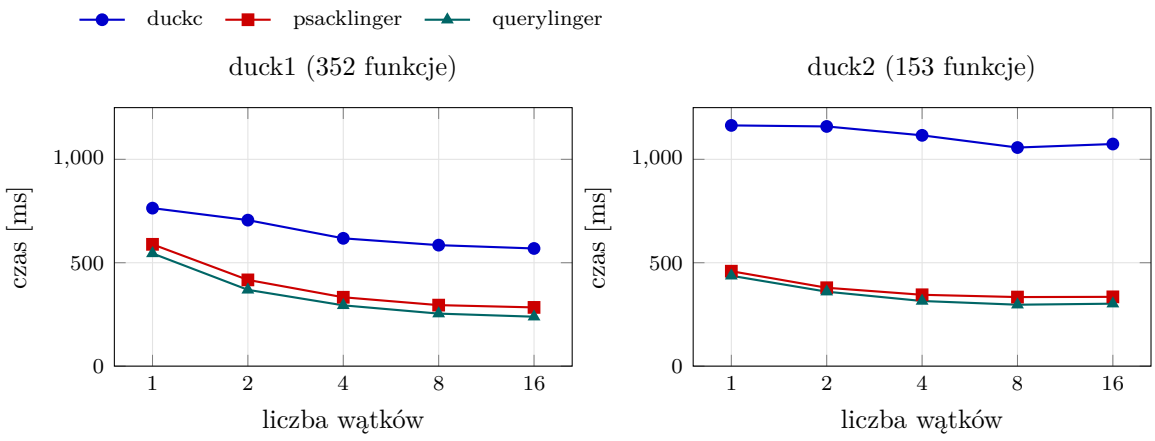
5.1 Pełna kompilacja

Pierwszy pomiar to kompilacja całego pliku od zera, bez korzystania z jakichkolwiek wcześniejszych wyników. Zbiera go tabela 2, a rysunek 3 pokazuje te same liczby jako krzywe skalowania.

Tabela 2: Czas pełnej kompilacji w zależności od liczby wątków roboczych. Dla `duckc` liczba ta oznacza liczbę wątków frameworka zapytań, dla pozostałych liczbę wątków roboczych kompilatora.

	duck1 (352 funkcje), czas [ms]			**duck2** (153 funkcje), czas [ms]		
wątki	duckc	psacklinger	querylinger	duckc	psacklinger	querylinger
1	764	589	545	1164	459	437
2	706	417	369	1159	379	360
4	618	333	294	1116	345	315
8	585	295	254	1057	334	297
16	569	284	**239**	1074	335	**302**

Punkt odniesienia bez frameworka: duckc jednowątkowo 625 / 1070 ms, psacklinger sekwencyjnie 588 / 461 ms.



Rysunek 3: Skalowanie czasu pełnej kompilacji wraz z liczbą wątków roboczych, dla obu plików wejściowych. Oś pozioma w skali logarytmicznej o podstawie 2.

Oba nasze kompilatory biją `duckc` przy każdej liczbie wątków: około 2,4 raza na `duck1` i około 3,6 raza na `duck2` przy ośmiu wątkach. Jak zaznaczyliśmy w sekcji 3, duckc robi przy tym nieporównanie więcej, a jego czas w dużej części zajmują fazy, które muszą wykonać się jedna po drugiej: budowa drzewa składniowego, a potem łańcuch zapytań rozwiązujących nazwy, w którym kolejne zapytanie czeka na poprzednie. Nasze kompilatory dzielą pracę na pojedyncze funkcje i takiego łańcucha nie mają.

Querylinger wypada lepiej od psacklingera przy każdej liczbie wątków, o 5–20%. Pula zadań frameworka Query nie kosztuje przy tej granularności więcej niż osobne obliczenia PSAC dla każdej funkcji.

Osobnym pytaniem było, dlaczego `duckc` prawie nie zyskuje na dodatkowych wątkach. Przyczyną okazała się budowa drzewa składniowego, czyli parsowanie wraz z liczeniem stabilnych hashy SHA-256 dla każdego elementu, zajmuje około 232 ms i jest w całości szeregowa, co stanowi mniej więcej połowę czasu przy ośmiu wątkach. Zrównoleglenie liczenia tych hashy jest tu największą pojedynczą dźwignią: eksperyment z całkowitym wyłączeniem hashowania skrócił samo parsowanie z 202 ms do 67 ms.

Rozbicie czasu psacklingera na fazy (`duck1`, osiem wątków) pokazuje, gdzie naprawdę schodzi czas: parsowanie 67 ms, rejestracja deklaracji globalnych 1 ms, zbieranie jednostek pracy wraz z hashowaniem 22 ms, emisja obiektu ze zmiennymi globalnymi 2 ms, generowanie kodu funkcji 58 ms, linkowanie 55 ms. Faza faktycznie zrównoleglona to więc mniej niż połowa całości, co jest ważnym kontekstem dla wszystkich powyższych porównań.

Osobnej uwagi wymaga sam PSAC. Jego scheduler tworzy pulę wątków jeszcze przed wejściem do funkcji `main`, a wolne wątki kręcą się w pętli zamiast spać, przez co szesnaście z nich psuło zawartość pamięci podręcznej przez całą szeregową fazę front endu. Zatrzymanie puli na jednym wątku poza fazami równoległymi skróciło pełną kompilację mniej więcej o połowę.

5.2 Aktualizacje inkrementalne

Drugi pomiar dotyczy tego, po co w ogóle buduje się silnik inkrementalny: czasu reakcji na pojedynczą edycję. Kompilatory pracowały w trybie interaktywnym, przy ośmiu wątkach, a edycje były jednoliniowe. Wyniki zbiera tabela 3.

Tabela 3: Czas aktualizacji inkrementalnej. W każdej komórce podano średni czas w milisekundach oraz liczbę faktycznie przekompilowanych funkcji.

	duck1 (352 funkcje)		**duck2** (153 funkcje)	
scenariusz	psacklinger	querylinger	psacklinger	querylinger
	czas [ms] / funkcje		czas [ms] / funkcje	
brak zmian	109,0 / 0	118,4 / 0	107,2 / 0	106,4 / 0
edycja ciała funkcji	149,8 / 1	160,1 / 1	140,0 / 1	146,3 / 1
zmiana inicjalizatora globala	155,9 / 0	163,7 / 0	140,2 / 0	147,7 / 0

Około 100 ms to podłoga, poniżej której nie da się zejść, bo tyle zajmuje wspólny front end, który celowo *nie* jest inkrementalny i przy każdej aktualizacji parsuje oraz analizuje program od nowa. Jest on identyczny w obu kompilatorach i został zostawiony poza grafem zależności właśnie po to, żeby pomiar dotyczył samych silników.

Przy równej granularności zależności wyników obu silników nie da się od siebie odróżnić. Psacklinger jest minimalnie szybszy przy aktualizacji, querylinger przy pełnej kompilacji, ale różnice rzędu kilku milisekund mieszczą się w odchyleniach tego samego rzędu, więc nie traktujemy ich jako rozstrzygnięcia.

Więcej mówi porównanie tych liczb z wcześniejszą wersją psacklingera. Jego pierwsza wersja śledziła zależności tylko na poziomie całego programu, więc przy zmianie inicjalizatora zmiennej globalnej przekompilowywała 352 funkcje z 352, w czasie około 220 ms. Przejście na zależności śledzone dla pojedynczych symboli zbiło ten scenariusz do zera przekompilowanych funkcji. Efekt tej jednej zmiany jest o rząd wielkości większy od wszystkich różnic, jakie zmierzyliśmy między

silnikami.

5.3 Granularność zależności

Oba silniki doprowadziliśmy do tej samej, sprawdzonej testami granularności. Zbiera ją tabela 4. Osiągnęły ją innymi środkami: w PSAC każdy symbol dostaje własną modyfikowalną komórkę, a w Query własny side input. W obu przypadkach odpowiada za to wspólny mechanizm w generatorze kodu: za każdym razem, gdy generator sięga po stałą, sygnaturę funkcji, układ pól klasy albo zmienną globalną, zgłasza to silnikowi inkrementalnemu, a ten zapisuje zależność. Zbiór zależności nie jest więc deklarowany z góry, tylko powstaje jako produkt uboczny wykonania, dokładnie tam, gdzie symbol jest naprawdę używany.

Tabela 4: Granularność zależności. Wyniki identyczne dla obu silników, potwierdzone testami sprawdzającymi dokładne liczby przekompilowanych funkcji.

Rodzaj edycji	**Co zostaje przekompilowane**
zmiana ciała funkcji	tylko ta funkcja
zmiana wartości stałej	tylko funkcje, które ją czytają
zmiana inicjalizatora zmiennej globalnej	zero funkcji, tylko ponowna emisja obiektu ze zmiennymi globalnymi i relink
zmiana nazwy parametru	tylko ta funkcja, bo hashe sygnatur obejmują typy, a nie nazwy
zmiana typu zwracanego	funkcja oraz jej wywołujący, którzy poprawnie zgłaszają błąd typów
dodanie pola do klasy	tylko funkcje używające tej klasy
dodanie lub usunięcie funkcji	bez pełnej przebudowy, reszta zostaje w cache'u
nieudana kompilacja, przyczyna później naprawiona	ponowienie dokładnie tej funkcji, przez zależność od hasha całego programu
zmiana importu lub przestrzeni nazw	wszystko, jako bezpieczne rozwiązanie zachowawcze

Ostatnie dwa wiersze wymagają komentarza. Zmiana importu albo nazwy przestrzeni nazw może zmienić to, do czego odnosi się dowolna nazwa w programie, więc jedyną poprawną reakcją jest przekompilowanie wszystkiego. Podobnie zgrubna musi być reakcja na błąd kompilacji: nieudana kompilacja nigdy nie poznała swojego prawdziwego zbioru zależności, bo brakujący symbol nie ma jak pojawić się w grafie, więc jedyne, na czym może się zawiesić, to hash całego programu.

6 Optymalizacje kompilatora Duckling

Wnioski z projektu wykorzystaliśmy do optymalizacji kompilatora `duckc`. Wąskich gardeł szukaliśmy profilerem `perf`, nagrywając przebiegi kompilacji i patrząc, gdzie schodzi czas.

Pierwszą zmianą była zamiana alokatora pamięci na mimalloc. Wpadliśmy na to, patrząc na konfigurację budowania PSAC-a, który go używa: pojawiło się pytanie, ile z jego lekkości bierze się z modelu obliczeń, a ile z tańszych alokacji. Zmiana dała powtarzalne przyspieszenie od 10% do nawet 50%. Zużycia pamięci nie zmierzyliśmy.

Po włączeniu mimalloc profil się przesunął i nowym wąskim gardłem okazało się internowanie napisów, czyli mechanizm, który trzyma jedną kopię każdego identyfikatora. Chronił go globalny rwlock, przy czym większość czasu schodziła nie na blokadzie zapisu, ale na blokadzie odczytu na szybkiej ścieżce, czyli w przypadku, w którym identyfikator już istniał. Zastąpienie tego globalnym mutexem w połączeniu ze współbieżnymi hashmapami dało 10–20% w scenariuszu wielowątkowym. Pojedyncze założenie blokady odczytu jest tanie, ale przy wielu wątkach staje się punktem, o który wszystkie one konkurują.

Trzeci obszar to hashmapy. Używamy własnej implementacji współbieżnej hashmapy z własnym alokatorem. Sprawdziliśmy alternatywę w postaci Google Swiss Table, ale w naszym sposobie użycia okazała się wolniejsza. Zmierzyliśmy też, że zły dobór algorytmu haszującego przy wielu wątkach dodatkowo powoduje przerzucanie linii pamięci podręcznej między rdzeniami (cache-line bouncing), czyli koszt, którego w ogóle nie widać w pomiarach jednowątkowych.

Zapisaliśmy też kierunki, których nie zdążyliśmy zrobić: optymalizacja wykrywania cykli w grafie zapytań, przyspieszenie haszowania wyników, dalsza walka z przerzucaniem linii pamięci podręcznej oraz wczytywanie danych z wyprzedzeniem (software prefetching).

7 Dyskusja i ograniczenia

Benchmarki syntetyczne a rzeczywistość (RQ3). Benchmarki syntetyczne przewidywały wyraźną przewagę Query na gęstych grafach, sięgającą 2,5 raza przy prawdziwej pracy. W kompilatorze wyszedł praktycznie remis: querylinger jest szybszy o 5–20% przy pełnej kompilacji, psacklinger minimalnie szybszy przy aktualizacji inkrementalnej.

Wynika to naszym zdaniem stąd, że benchmark syntetyczny z workloadem zero mierzy stosunek samych narzutów, a kompilator nie pracuje w takich warunkach: wygenerowanie kodu jednej funkcji to milisekundy pracy, wobec których koszt zapisania kilkunastu zależności jest pomijalny. Do tego duża część czasu leży poza silnikiem inkrementalnym: w psacklingerze parsowanie i linkowanie to razem około 122 ms z 295 ms przy ośmiu wątkach. Benchmark syntetyczny mierzy więc głównie tę część kosztu, która w kompilatorze ma najmniejszy udział.

Wpływ granularności zależności (RQ4). Ten wynik uważamy za najważniejszy w całym projekcie. Przy równej granularności zależności dwa bardzo różne silniki dały wyniki nie do odróżnienia w granicach błędu pomiaru. Za to zmiana granularności w obrębie jednego silnika, ze śledzenia zależności na poziomie całego programu na śledzenie pojedynczych symboli, zmieniła jeden ze scenariuszy z 352 przekompilowanych funkcji w 220 ms na zero funkcji. Sugeruje to, że przy budowie kompilatora inkrementalnego warto zająć się najpierw precyzją śledzenia zależności oraz zmniejszaniem tej części pracy, która zostaje poza grafem, a wyborem silnika dopiero później.

Przypadki, w których silnik ma znaczenie. Modele nie są jednak wymienne. Przy bardzo lekkich węzłach obliczenia i płytkich zależnościach narzut Query jest nie do zignorowania, w skrajnym przypadku aż 17-krotny, a przy gęstych zależnościach i wielu wątkach PSAC przestaje skalować. Kompilator mieści się pomiędzy tymi przypadkami i prawdopodobnie dlatego wyniki wyszły tak zbliżone.

Ograniczenia i zagrożenia dla trafności. Wyniki trzeba czytać z następującymi zastrzeżeniami:

- Benchmarki syntetyczne są celowo dopasowane do przeciwnych modeli, więc podane stosunki to wartości skrajne, a nie typowe.
- Wszystkie pomiary pochodzą z jednej maszyny i jednej serii pomiarowej. Nie przypisywaliśmy wątków do rdzeni ani nie kontrolowaliśmy taktowania procesora, a procesor ma rdzenie o różnej wydajności, co przy szesnastu wątkach może zaburzać porównania.
- Korpus kompilatorów pochodzi z jednego generatora i obejmuje dwa pliki. Nie wiemy, na ile wygenerowany kod przypomina strukturę zależności prawdziwego projektu.
- Nieinkrementalny front end daje podłogę około 100 ms, która zasłania różnice między silnikami przy aktualizacjach. Do zmierzonego czasu wliczone jest też linkowanie i operacje dyskowe, na które żaden silnik nie ma wpływu.
- Wyniki ArraySum są uśrednione po dwóch rozmiarach tablicy, rozmiar nie jest osobną osią wykresu. Dodatkowo przy zerowym workloadzie wzrost procentu aktualizacji z 1 do 100 zmienia czas PSAC-a tylko z 0,10 s na 0,12 s. Nie udało nam się tego wyjaśnić i nie wykluczamy, że to artefakt pomiaru.
- Nie zmierzyliśmy zużycia pamięci przez kompilatory, mimo że oba silniki mają struktury, które w długiej sesji interaktywnej tylko rosną: tablica komórek i wpisy po usuniętych symbolach po stronie PSAC-a, węzły grafu po stronie Query.

- Nie robiliśmy testów istotności statystycznej. Przy różnicach rzędu kilku milisekund i odchyleniach tego samego rzędu właśnie dlatego nie rozstrzygamy porównania przy aktualizacjach.

8 Wnioski i dalsze prace

Wracamy na koniec do pytań postawionych we wstępie.

RQ1. Mniejszy narzut własny ma PSAC. Na czystym narzucie jest lżejszy w obu benchmarkach syntetycznych, wyraźnie w ArraySum (4–5 razy) i umiarkowanie w DAG-u (1,2–1,6 raza).

RQ2. Query skaluje się stabilniej. PSAC radzi sobie dobrze na płytkim drzewie ArraySum, ale na gęstym DAG-u jego czas potrafi rosnąć wraz z liczbą wątków. W kompilatorze oba skalują podobnie i wyraźnie lepiej niż `duckc`.

RQ3. Tylko częściowo. Przewidywany kierunek różnicy się potwierdził, ale w prawdziwym kompilatorze okazała się ona znacznie mniejsza: zamiast 2,5 raza mamy 5–20% przy pełnej kompilacji i różnicę nieistotną przy aktualizacjach.

RQ4. Większe znaczenie ma granularność. Przy jednakowej granularności wyników obu silników nie da się odróżnić, a jej poprawa w obrębie jednego silnika przyniosła efekt o rząd wielkości większy.

Dla kompilatora Duckling oznacza to, że nie widzimy powodu do zmiany frameworka. Query jest już zintegrowany z resztą potoku i przy tej wielkości zadań nie kosztuje więcej niż PSAC, więc dalszy wysiłek lepiej kierować na precyzję zależności i na fazy poza grafem.

Wśród dalszych prac widzimy przede wszystkim uczynienie front endu inkrementalnym, co usunęłoby podłogę 100 ms i pozwoliłoby zmierzyć same silniki; zrównoleglenie liczenia stabilnych hashy w `duckc` (skrócenie parsowania z 202 ms do 67 ms); pomiar zużycia pamięci obu silników; oraz dokończenie listy z sekcji 6.

9 Podział pracy

Tabela 5: Podział pracy w projekcie.

Osoba	**Zakres prac**
Jakub Bartmiński	Analiza modelu PSAC wraz z wizualizacją jego działania. Implementacja i pomiary benchmarków syntetycznych oraz skrypt rysujący z nich mapy cieplne. Poprawki i rozszerzenia psacklingera, w tym rozbudowa testów.
Piotr Oleszczuk	Implementacja psacklingera: instrukcje sterujące, klasy, przypisania, dostęp do pól i testy. Implementacja, parametryzacja i pomiary benchmarków syntetycznych.
Piotr Trzaskowski	Analiza frameworka Query. Optymalizacje `duckc`: profilowanie, zmiana alokatora pamięci, usunięcie globalnej blokady przy internowaniu napisów i praca nad hashmapami. Implementacja psacklingera.

Bibliografia

[1] Daniel Anderson i in. “Efficient Parallel Self-Adjusting Computation”. W: *Proceedings of the 33rd ACM Symposium on Parallelism in Algorithms and Architectures (SPAA ’21)*. ACM, 2021. DOI: 10.1145/3409964.3461799.